IEEE 830 / ISO-IEC-IEEE 29148 STANDARD DOCUMENT ID: SG-SRS-2026-V1.0

ShadowGestures: Invisible Edge Gesture Launcher

Software Requirements Specification (SRS), UML Architecture, UI/UX Wireframes & Kotlin Core Scaffolding

< 500 ms	20 × 160 dp	0% Network	< 1.2% / 24h
End-to-End Launch SLA	Default Right Edge Zone	100% Offline Privacy	Idle Battery Ceiling

1. Introduction, Purpose & Project Objectives

SECTION 01

ShadowGestures is a lightweight, background-resident Android utility that replaces multi-step home screen and app drawer navigation with an invisible, zero-clutter gesture trigger. By swiping inward from a dedicated 20×160 dp transparent screen-edge region and sketching a custom unistroke symbol, users immediately launch bound applications or system shortcuts. This specification establishes the complete functional, architectural, UI/UX, and code-level foundation for engineering execution.

- **Sub-500ms Execution Pipeline:** Guarantee that coordinate resampling, spatial normalization, template matching, and Android intent dispatch execute in under 500 ms from finger lift (`ACTION_UP`), with mathematical classification completing in under 15 ms.
- **Zero Visual & Navigational Interference:** Coexist harmoniously with Android 10+ gesture navigation using targeted system gesture exclusion APIs and non-blocking window flags so underlying typing, scrolling, and gaming remain unobstructed.
- **Strict Offline Privacy & Battery Discipline:** Operate without internet permissions (zero telemetry) and suspend touch listeners automatically upon `ACTION_SCREEN_OFF` to eliminate standby wake-locks.

2. Project Scope & Boundary Definitions

SECTION 02

2.1 In-Scope Deliverables (v1.0)

- Event-driven `AccessibilityService` lifecycle bound to device unlock (`ACTION_USER_PRESENT`) and screen sleep states.
- Configurable transparent edge trigger (Left/Right edge, adjustable width 10–40 dp, height 80–200 dp, vertical offset).
- Full-screen hardware-accelerated gesture overlay canvas with real-time stroke rendering and haptic feedback.
- Offline **$1 Unistroke Recognizer** engine supporting 64-point resampling and scale/rotation invariance.
- Local Room SQLite persistence mapping custom symbol templates to installed application package names.

2.2 Out-of-Scope & Future Roadmap

- **Cloud Synchronization (Out of Scope):** No remote account backup or network permissions to preserve cryptographic privacy guarantees.
- **Lock-Screen Bypass (Out of Scope):** Gestures cannot bypass Android Keyguard authentication; activation strictly requires an unlocked session.
- **Deep In-App Macro Scripting (v2.0 Roadmap):** v1.0 launches top-level application activities; automated UI clicking sequences are deferred.

3. Three-Phase Operational Workflow

SECTION 03

Phase	System Trigger & State	Android OS Subsystem Behaviour	Latency Target
Phase 1: Unlock	User passes biometric/PIN Keyguard (`ACTION_USER_PRESENT`).	Background service wakes from 0% CPU idle and attaches the 20×160 dp invisible edge view to `WindowManager`.	< 30 ms attach
Phase 2: Edge Swipe	Inward horizontal swipe inside the 20×160 dp right-edge zone.	`setSystemGestureExclusionRects` suppresses OS Back gesture; `deltaX < -24px` triggers canvas expansion.	< 16 ms (1 frame)

4. Functional Requirements Specification

SECTION 04

Req ID	Feature Module	Technical Requirement Specification	Priority
FR-01	Unlock Lifecycle	System shall register a dynamic receiver for `ACTION_USER_PRESENT` and `ACTION_SCREEN_OFF` to mount and unmount the edge overlay dynamically.	`P0-Crit`
FR-02	Edge Trigger Zone	System shall render a transparent view defaulting to Right Edge, 20 dp width, 160 dp height, centered vertically using `FLAG_NOT_FOCUSABLE`.	`P0-Crit`
FR-03	Nav Conflict Guard	System shall invoke `ViewCompat.setSystemGestureExclusionRects()` bounded to the 160 dp edge height (under Android's 200 dp limit) to prevent Back-swipe collisions.	`P0-Crit`
FR-04	Inward Swipe Filter	Edge zone shall ignore vertical scroll touches and trigger the drawing overlay strictly when horizontal inward displacement exceeds 24 dp with `|dx| > 1.5 * |dy|`.	`P0-Crit`
FR-05	Drawing Overlay	Upon trigger, a `MATCH_PARENT` canvas overlay shall capture `MotionEvent` coordinates and render a smooth anti-aliased stroke trail at 60/120 Hz.	`P0-Crit`
FR-06	$1 Pattern Engine	Upon `ACTION_UP`, captured points shall be resampled to `N = 64` equidistant points, rotated to indicative angle 0°, scaled to a 250×250 square, and scored.	`P0-Crit`
FR-07	Confidence Gate	Matches scoring ≥ `0.80` (80% similarity) shall immediately launch the bound package; scores < `0.80` shall trigger a reject haptic pulse and dismiss cleanly.	`P0-Crit`
FR-08	Custom CRUD Store	Users shall record, test, rebind, and delete custom gesture symbols mapped to any launchable application via an offline Room database.	`P1-High`
FR-09	Landscape Auto-Pause	System shall optionally suspend the edge trigger when `ORIENTATION_LANDSCAPE` is detected to prevent accidental triggers during full-screen gaming.	`P1-High`

5. Non-Functional Requirements & SLA Benchmarks

SECTION 05

To ensure ShadowGestures feels like a native OS subsystem rather than a third-party utility, strict non-functional envelopes govern latency, memory footprint, power consumption, and security architecture.

> **Sub-500ms End-to-End Execution Budget Breakdown**
>
> **Touch-Up to App Transition Total: ≤ 500 ms.** Allocated component budgets: (1) Path Resampling & Normalization: **3.5 ms**; (2) Euclidean Matching across 50 stored templates: **4.5 ms**; (3) Overlay View Tear-down & Hardware Layer Release: **12.0 ms**; (4) Android `PackageManager` Intent Resolution & `ActivityTaskManager` Warm/Cold Start Handshake: **220–450 ms**.

6. Security, Privacy & OS Compatibility Constraints

SECTION 06

- **Zero-Network Attack Surface (NFR-SEC-01):** The `AndroidManifest.xml` explicitly omits `android.permission.INTERNET` and `ACCESS_NETWORK_STATE`. It is physically impossible for gesture coordinates or app usage habits to leave the device.
- **Overlay Tapjacking Protection (NFR-SEC-02):** While the edge zone is idle, its 20×160 dp bounds pass outside touches straight through via `FLAG_NOT_TOUCH_MODAL`. During active drawing, the overlay consumes touches strictly for stroke capture and auto-dismisses after 1,500 ms of inactivity.
- **Memory & Battery Envelope (NFR-PERF-01):** Resident RAM footprint in background service mode shall remain below **18 MB PSS**. Zero background polling threads or timers are permitted; execution is 100% interrupt-driven via

7. System Architecture & UML Sequence Diagrams

SECTION 07

ShadowGestures separates presentation overlays from mathematical recognition using a clean three-tier reactive architecture: the **OS Window & Lifecycle Layer**, the **Domain Recognition Engine**, and the **Room Persistence Repository**.

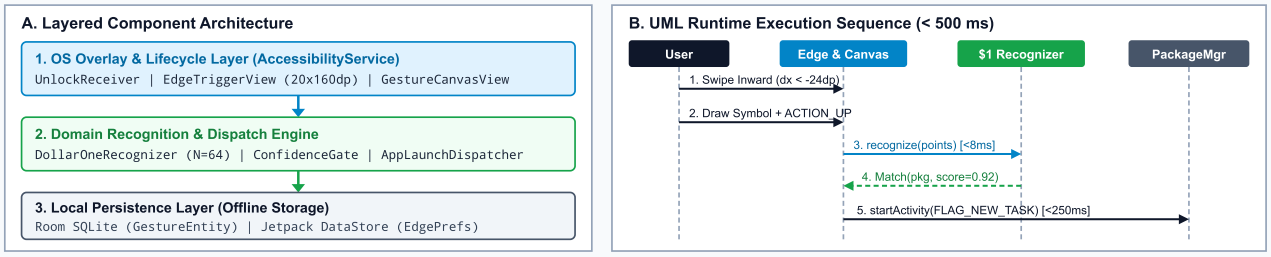


Figure 7.1 — Three-Tier Modular Architecture (Left) and Sub-500ms Runtime Sequence Pipeline (Right)

8. Mathematical Model: $1 Unistroke Recognizer & Database Schema

SECTION 08

Heavy machine learning frameworks (such as TensorFlow Lite) introduce 30–80 ms JNI initialization overhead and add 15+ MB to the APK size. ShadowGestures instead implements the geometric **$1 Unistroke Recognizer** algorithm in pure Kotlin, achieving rotation-, scale-, and position-invariant classification in under 5 ms.

Four-Step Geometric Normalization & Scoring Equations

Given a raw touch stroke P of variable length, the engine computes: (1) **Equidistant Resampling** into $N = 64$ points spaced at interval $I = PathLength(P) / (N - 1)$; (2) **Indicative Angle Alignment** by rotating around centroid $C = (\bar{x}, \bar{y})$ by $-\omega = -atan2(C_y - P_{0,y}, C_x - P_{0,x})$; (3) **Non-Uniform Scaling & Translation** to reference square $S = 250.0$ with centroid at origin $(0, 0)$; and (4) **Confidence Score** against stored template T via Golden Section Search:

$$\textbf{Score(P, T)} = 1 - \left[d_{min}(P, T) / (0.5 \times \sqrt{(S^2 + S^2)}) \right] \quad \text{where} \quad d(P, T) = (1 / N) \times \sum \sqrt{((P_{i,x} - T_{i,x})^2 + (P_{i,y} - T_{i,y})^2)}$$

8.1 Local Persistence Schema (Room SQLite & DataStore)

- **Table `gesture_mappings`:** `id: Long (PK, Auto)` | `gestureLabel: String` (e.g., "W - WhatsApp") | `targetPackage: String` (indexed) | `targetActivity: String?` | `normalizedPointsJson: String` (serialized array of 64 normalized `(x,y)` floats) | `createdAt: Long`.
- **Preferences `edge_config.preferences_pb`:** `edgeSide: Enum(RIGHT, LEFT) = RIGHT` | `widthDp: Int = 20` | `heightDp: Int = 160` | `verticalBiasPercent: Int = 50` (Centered) | `minConfidence: Float = 0.80f` | `pauseInLandscape: Boolean = true`.

9. UI/UX Screen Wireframes & Interaction Design

SECTION 09

While ShadowGestures operates invisibly during daily use, it includes a clean Material 3 configuration console for tuning edge geometry, inspecting saved gesture bindings, and recording new unistroke templates.

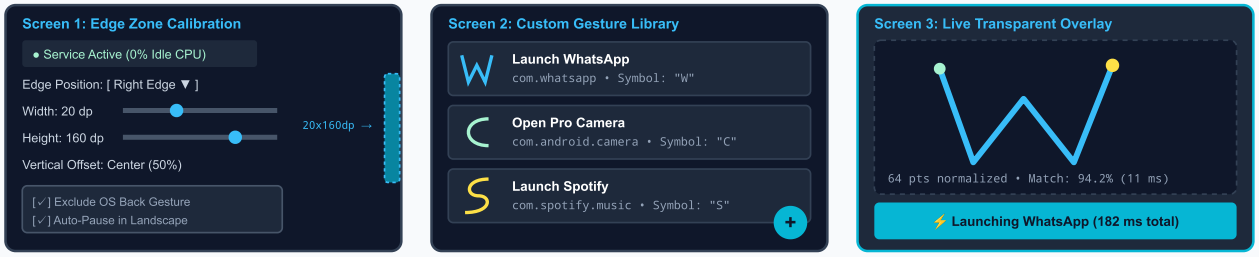


Figure 9.1 — Wireframes: (1) Edge Zone Calibration, (2) Custom Gesture Library, and (3) Active Overlay Execution

10. UI State Machine & Haptic Micro-Interactions

SECTION 10

Overlay UI State	Entry Condition	Visual & Window Behavior	Haptic Feedback
1. DORMANT	Screen off (`ACTION_SCREEN_OFF`) or locked.	Edge view detached from `WindowManager`; 0% CPU usage.	None
2. ARMED_INVISIBLE	Device unlocked (`ACTION_USER_PRESENT`).	20×160 dp right-edge view active at `alpha = 0.0f` (fully transparent).	None
3. CANVAS_DRAWING	Inward swipe crosses `-24` dp threshold.	Full-screen scrim (#33000000) mounts; cyan path trails finger at 120 Hz.	`CLOCK_TICK` (Crisp)
4. MATCH_LAUNCH	`ACTION_UP` with best score ≥ `0.80`.	Canvas removes immediately; target app launches with `FLAG_ACTIVITY_NEW_TASK`.	`CONFIRM`
5. REJECT_DISMISS	`ACTION_UP` with score < `0.80` or < 8 pts.	Path flashes coral red (#F43F5E) for 120 ms and dissolves cleanly.	`REJECT` (Double pulse)

11. Custom Gesture Onboarding & Anti-Collision Validation

SECTION 11

- **3-Sample Averaging Recorder:** When creating a new gesture in Screen 2, the user sketches the symbol 3 times. The engine validates internal consistency (requiring ≥ 85% mutual similarity across the 3 samples) and stores the centroid-averaged 64-point template.
- **Ambiguity & Collision Guard:** Before saving a new gesture, the engine scores it against all existing templates in Room. If a new symbol scores ≥ `0.82` against an existing binding (e.g., drawing a "0" when "O" is already bound), the UI warns the user to pick a distinct shape or overwrite the existing binding.

12. Production Kotlin Scaffolding: Edge Service & $1 Engine

SECTION 12

The following two self-contained Kotlin modules implement the core technical mechanisms of ShadowGestures: the conflict-free 20×160 dp invisible edge overlay service and the sub-10ms **$1 Unistroke Recognizer** mathematical engine.

12.1 Accessibility Service, 20×160 dp Edge Trigger & System Gesture Exclusion

```kotlin
package com.shadowgestures.core.overlay

import android.accessibilityservice.AccessibilityService
import android.content.*
import android.graphics.*
import android.os.Build
import android.view.*
import androidx.core.view.ViewCompat
import kotlin.math.abs

class ShadowEdgeService : AccessibilityService() {
    private lateinit var windowManager: WindowManager
    private var edgeTriggerView: View? = null
    private var drawingCanvasView: GestureOverlayCanvas? = null

    private val unlockReceiver = object : BroadcastReceiver() {
        override fun onReceive(ctx: Context, intent: Intent) {
            when (intent.action) {
                Intent.ACTION_USER_PRESENT -> attachInvisibleEdgeZone()
                Intent.ACTION_SCREEN_OFF -> detachAllOverlays() // 0% idle battery while
sleeping
            }
        }
    }

    override fun onServiceConnected() {
        windowManager = getSystemService(WINDOW_SERVICE) as WindowManager
        registerReceiver(unlockReceiver, IntentFilter().apply {
            addAction(Intent.ACTION_USER_PRESENT)
            addAction(Intent.ACTION_SCREEN_OFF)
        })
        attachInvisibleEdgeZone()
    }

    private fun attachInvisibleEdgeZone() {
        if (edgeTriggerView != null) return
        val density = resources.displayMetrics.density
        val widthPx = (20 * density).toInt()   // Default: 20 dp width
        val heightPx = (160 * density).toInt() // Default: 160 dp height (< 200dp OS limit)
        val swipeSlopPx = 24f * density

        val params = WindowManager.LayoutParams(
            widthPx, heightPx,
            WindowManager.LayoutParams.TYPE_ACCESSIBILITY_OVERLAY,
            WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
                WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL or
                WindowManager.LayoutParams.FLAG_LAYOUT_IN_SCREEN,
            PixelFormat.TRANSLUCENT
        ).apply { gravity = Gravity.END or Gravity.CENTER_VERTICAL } // Right Edge, Centered

        edgeTriggerView = object : View(this) {
            private var startX = 0f; private var startY = 0f
            override fun onLayout(changed: Boolean, l: Int, t: Int, r: Int, b: Int) {
                super.onLayout(changed, l, t, r, b)
                if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
                    ViewCompat.setSystemGestureExclusionRects(this, listOf(Rect(0, 0, width,
```